LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 REKURSIF

Sigit Rohmatulloh

109082500185

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Definisi: Rekursif adalah teknik di mana sebuah subprogram memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kecil.

Komponen Utama:

- **Base-case (Basis):** Kondisi berhenti agar tidak terjadi perulangan tanpa henti (*infinite loop*).
- **Recursive-case:** Bagian di mana fungsi memanggil dirinya sendiri kembali.

Alur Eksekusi:

1. **Forward (Maju):** Melakukan pemanggilan ke dalam sampai mencapai *base-case*.
2. **Backward (Mundur):** Kembali ke pemanggilan sebelumnya untuk menyelesaikan instruksi yang tersisa.

Fungsi: Teknik ini merupakan alternatif dari struktur kontrol perulangan (*looping*).

B. Guided

1. Membuat baris bilangan dari n hingga 1

Week6_Modul5 > Guided > Guided 1 > baris_rekursif.go

```go
package main
import "fmt"
func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Print(bilangan)
        baris(bilangan - 1)
    }
}
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 1> go run baris_rekursif.go
5
54321
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 1> go run baris_rekursif.go
7
7654321
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 1>
```

Penjelasan:

Tujuan: Menampilkan barisan bilangan bulat menurun dari n sampai 1.

Metode: Menggunakan **rekursif**, yaitu subprogram yang memanggil dirinya sendiri.

Komponen Utama:

Base-case: Kondisi bilangan == 1 untuk menghentikan proses.

Recursive-case: Mencetak angka, lalu memanggil fungsi kembali dengan nilai n-1

Alur: Program melakukan proses **forward** (maju) hingga mencapai batas berhenti (*base-case*).

2. Menghitung hasil penjumlahan 1 hingga n

Week6_Modul5 > Guided > Guided 2 > penjumlahan_rekursif.go > ...

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}

```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 2> go run penjumlahan_rekursif.go
5
15
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 2> go run penjumlahan_rekursif.go
2
3
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 2> go run penjumlahan_rekursif.go
6
21
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 2>
```

Penjelasan:

Tujuan: Menghitung total hasil penjumlahan angka dari 1 sampai dengan n.
Metode: Menggunakan fungsi rekursif yang mengembalikan nilai (*return value*) berupa bilangan bulat (integer).
Komponen Utama:
Base-case: Kondisi n == 1 yang akan mengembalikan nilai 1 untuk menghentikan rekursi.
Recursive-case: Mengembalikan nilai n ditambah dengan hasil pemanggilan fungsi itu sendiri dengan parameter n – 1.
Alur: Program melakukan pemanggilan maju (*forward*) hingga mencapai angka 1, kemudian melakukan proses hitung mundur (*backward*) untuk menjumlahkan semua angka tersebut.

3. Mencari dua pangkat n atau 2^n

Week6_Modul5 > Guided > Guided 3 > pangkat_rekursif.go > pangkat

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return n * pangkat(n-1)
    }
}
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 3> go run pangkat_rekursif.go
pangkat_rekursif.go:1:1: expected 'package', found 'EOF'
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 3> go run pangkat_rekursif.go
4
24
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 3> go run pangkat_rekursif.go
6
720
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Guided\Guided 3>
```

Penjelasan:

Tujuan: Menghitung hasil perpangkatan dengan basis 2 dan eksponen n (2^n).

Metode: Menggunakan fungsi rekursif yang mengembalikan nilai integer

Komponen Utama:

Base-case: Kondisi n == 0 yang mengembalikan nilai 1 (2^0 = 1) sebagai titik henti rekursi.

Recursive-case: Mengembalikan hasil perkalian antara angka 2 dengan pemanggilan fungsi itu sendiri menggunakan parameter n - 1.

Alur: Program melakukan perkalian angka 2 secara berulang sebanyak n kali melalui tumpukan rekursi.

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan sn= Sn−1 + Sn−2 . Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban:

Week6_Modul5 > Unguided > Unguided 1 > fibonacci.go > fibonacci

```go
package main

import "fmt"

func main() {
    var n int

    fmt.Scan(&n)

    fmt.Println(fibonacci(n))
}

func fibonacci(n int) int {

    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {

        return fibonacci(n-1) + fibonacci(n-2)
    }
}
```

Output :

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 1> go run fibonacci.go
10
55
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 1> go run fibonacci.go
7
13
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 1>
```

Penjelasan :

Tujuan: Menghitung nilai suku ke-n pada deret Fibonacci.

Metode: Rekursif dengan rumus Sn = Sn-1 + Sn-2.

Komponen Utama:

Base-case: Kondisi jika n=0 (hasil 0) atau n=1 (hasil 1) sebagai penghenti rekursi.

Recursive-case: Menjumlahkan hasil dari dua pemanggilan fungsi sebelumnya (n-1 dan n-2).

Alur: Program akan membentuk struktur pohon (*tree*) di mana setiap pemanggilan akan terus bercabang hingga menyentuh angka 0 atau 1.

2. Soal Unguided 2

Buatlah sebuah program yang digunakan untuk menampilkan pola bintang berikut ini dengan menggunakan fungsi rekursif. N adalah masukan dari user

Jawaban :

Week6_Modul5 > Unguided > Unguided 2 > faktor_bilangan.go > cariFaktor

```go
package main

import "fmt"

func main() {
    var n int

    fmt.Scan(&n)

    cariFaktor(n, 1)
    fmt.Println()
}

func cariFaktor(n int, i int) {

    if i <= n {

        if n%i == 0 {
            fmt.Print(i, " ")
        }

        cariFaktor(n, i+1)
    }
}
```

Output :

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 2> go run faktor_bilangan.go
5
1 5
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 2> go run faktor_bilangan.go
12
1 2 3 4 6 12
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 2>
```

Penjelasan :

Tujuan: Menampilkan semua faktor bilangan dari N secara berurutan dari 1 hingga N.

Metode: Menggunakan rekursif untuk menguji setiap angka dari 1 sampai N apakah dapat membagi N tanpa sisa.

Komponen Utama:

Base-case: Rekursi berhenti ketika nilai pembagi (i) lebih besar dari nilai N.

Recursive-case: Mengecek kondisi n % i == 0, mencetak angka jika benar, lalu memanggil fungsi kembali dengan nilai i + 1.

3. Soal Unguided 3

   Buatlahprogram yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N

   Jawaban :

```
Week6_Modul5 > Unguided > Unguided 3 > ganjil_rekursif.go > cetakGanjil
package main

import "fmt"

func main() {
    var n int

    fmt.Scan(&n)

    cetakGanjil(n, 1)
    fmt.Println()
}

func cetakGanjil(n int, i int) {

    if i <= n {

        if i%2 != 0 {
            fmt.Print(i, " ")
        }

        cetakGanjil(n, i+1)
    }
}
```

   Output :

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 3> go run ganjil_rekursif.go
20
1 3 5 7 9 11 13 15 17 19
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 3> go run ganjil_rekursif.go
5
1 3 5
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 3> go run ganjil_rekursif.go
20
1 3 5 7 9 11 13 15 17 19
PS D:\109082500185_Sigit-Rohmatulloh\Week6_Modul5\Unguided\Unguided 3>
```

   Penjelasan :

   Tujuan: Menampilkan barisan bilangan ganjil mulai dari 1 hingga batas N yang ditentukan pengguna.

Metode: Menggunakan teknik **rekursif** untuk memeriksa setiap bilangan secara berurutan.

Komponen Utama:

Base-case: Rekursi akan berhenti ketika parameter pencacah (i) bernilai lebih besar dari N.

Recursive-case: Program memeriksa kondisi bilangan ganjil menggunakan operator modulo (i % 2 != 0), kemudian memanggil dirinya sendiri dengan menambahkan nilai pencacah (i + 1).

D. Kesimpulan

- **Rekursif** adalah teknik di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan tugas.

- **Base-case (Basis)** adalah bagian paling penting sebagai syarat agar program berhenti dan tidak jalan terus tanpa henti.

- **Recursive-case** adalah kondisi di mana fungsi memanggil dirinya sendiri kembali.

- Proses rekursif punya dua tahap: **Forward** (maju sampai batas berhenti) dan **Backward** (jalan mundur menyelesaikan sisa perintah).

- Rekursif adalah alternatif untuk menggantikan cara perulangan biasa (*looping*).